

RAPPORT DE RÉFLEXION

Benchmark DOM — React vs Vue vs Svelte

Yaovi Urbain · Juin 2026

1. Défis rencontrés

Le principal défi de ce projet était d'implémenter la même application de manière idiomatique dans trois frameworks aux philosophies très différentes. React impose une approche déclarative avec des hooks (`useState`, `useCallback`) et une immutabilité stricte des données — chaque modification crée un nouvel état plutôt que de muter l'existant. Cette contrainte, bien que bénéfique pour les performances grâce au Virtual DOM, demande une discipline de code rigoureuse.

Vue 3 avec la Composition API (`ref`, `reactive`) offre une syntaxe plus intuitive, mais la réactivité fine sur les objets imbriqués peut introduire un overhead inattendu, comme observé lors des mises à jour de 50 tâches (1.700 ms contre 0.200 ms pour React et Svelte). Comprendre quand Vue re-rend les composants et comment l'optimiser représentait un défi conceptuel.

Svelte, quant à lui, adopte une approche radicalement différente : la compilation à la construction élimine le Virtual DOM et génère du JavaScript natif optimisé. Bien que le code soit le plus simple à écrire, comprendre ses limites (comme la réactivité qui nécessite une réassignation explicite) a demandé un apprentissage spécifique.

2. Impact des approches sur les performances

Les résultats du benchmark révèlent comment chaque approche de mise à jour du DOM influence directement les performances. React utilise un Virtual DOM qui compare l'état précédent et le nouvel état pour calculer le minimum de modifications à appliquer au DOM réel (reconciliation). Cette approche est efficace pour les mises à jour groupées, ce qui explique ses 0.200 ms constants.

Vue 3 utilise également un Virtual DOM, mais son système de réactivité basé sur les Proxies JavaScript suit les dépendances de manière plus granulaire. Pour les mutations directes d'objets dans un tableau (comme la mise à jour de 50 tâches), Vue

doit parcourir et invalider les dépendances réactives une par une, ce qui explique le temps de 1.700 ms observé.

Svelte adopte l'approche la plus radicale : au lieu d'un Virtual DOM, le compilateur génère du code JavaScript qui met à jour chirurgicalement le DOM réel uniquement là où c'est nécessaire. C'est pourquoi la suppression de 50 tâches prend 0.000 ms — le code généré est aussi optimal que du JavaScript natif écrit à la main.

3. Meilleur framework par scénario

Scénario	Meilleur framework	Raison
Rendu initial (100 tâches)	Svelte	Pas de Virtual DOM, rendu direct
Rendu initial (500+ tâches)	Vue / Égalité	Proxy réactif efficace pour les grandes listes
Mises à jour fréquentes	React / Svelte	Virtual DOM optimisé vs code compilé
Suppressions	Svelte	Code natif généré, 0.000 ms
Facilité de développement	Vue / Svelte	Syntaxe plus intuitive
Écosystème et popularité	React	Communauté la plus large

En conclusion, Svelte démontre les meilleures performances brutes grâce à son approche de compilation, tandis que React offre le meilleur équilibre entre performance et écosystème. Vue reste un excellent choix pour sa courbe d'apprentissage douce, malgré un overhead légèrement supérieur sur certaines opérations de mise à jour.